Computer Vision
Midterm Project

Indrayudd Roy Chowdhury

This project is an intuitive spin on a classical problem of counting coins. The main goals were to make a robust coin-counting pipeline using classical CV concepts, and to convey the steps intuitively so that anyone can understand the steps the algorithm goes through to arrive at the final coin detection.

Counting Coins, Accessibly

Uncovers the Mathematical Elegance

Sample 6	28 pred
GT 29	all
dataset/train/train_2.jpg	

Sample 7	27 pred
GT 29	all
dataset/test/test_1.jpg	

Sample 8	30 pred
GT 31	all
dataset/train/train_25.jpg	

Sample 9	38 pred
GT 38	all
dataset/test/test_15.JPG	

Sample 10	50 pred
GT 50	all
dataset/10c/IMG_4191.JPG	

Steps

STEP 1

Original Frame

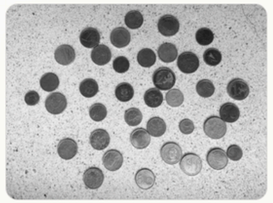
This is the untouched photo: what a person sees before any computer cleanup.

Open detailed inspector →

STEP 2

Grayscale Projection

Convert color into one brightness channel so boundaries become easier to compare.

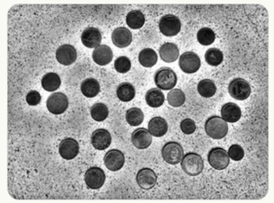


Open detailed inspector →

STEP 3

CLAHE Contrast Boost

Brighten dim local areas and tame over-bright zones so coin rims pop consistently.



Open detailed inspector →

STEP 4

Specular Suppression

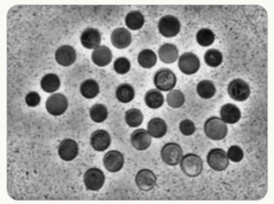
Mirror-like bright spots are repaired so circular coin borders are no longer broken.

Open detailed inspector →

STEP 5

Low-Pass Filtering

Fine background grain is softened so edges mostly belong to coins, not surface texture.

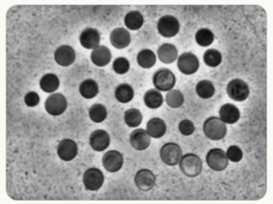


Open detailed inspector →

STEP 6

Line-Artifact Suppression

Long tablecloth or wood stripes are removed so circle detection is not distracted.

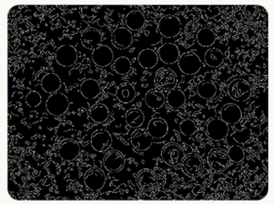


Open detailed inspector →

STEP 7

Canny Edge Map

Highlight strong object boundaries so circles can be detected from rim outlines.

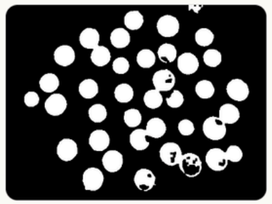


Open detailed inspector →

STEP 8

Foreground Segmentation

Convert the image into coin-like vs background areas to narrow where circles can exist.

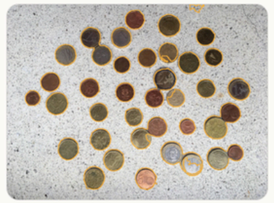


Open detailed inspector →

STEP 9

Region Boundary View

When coins touch, we split merged blobs into separate regions before final circle fitting.

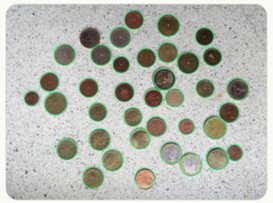


Open detailed inspector →

STEP 10

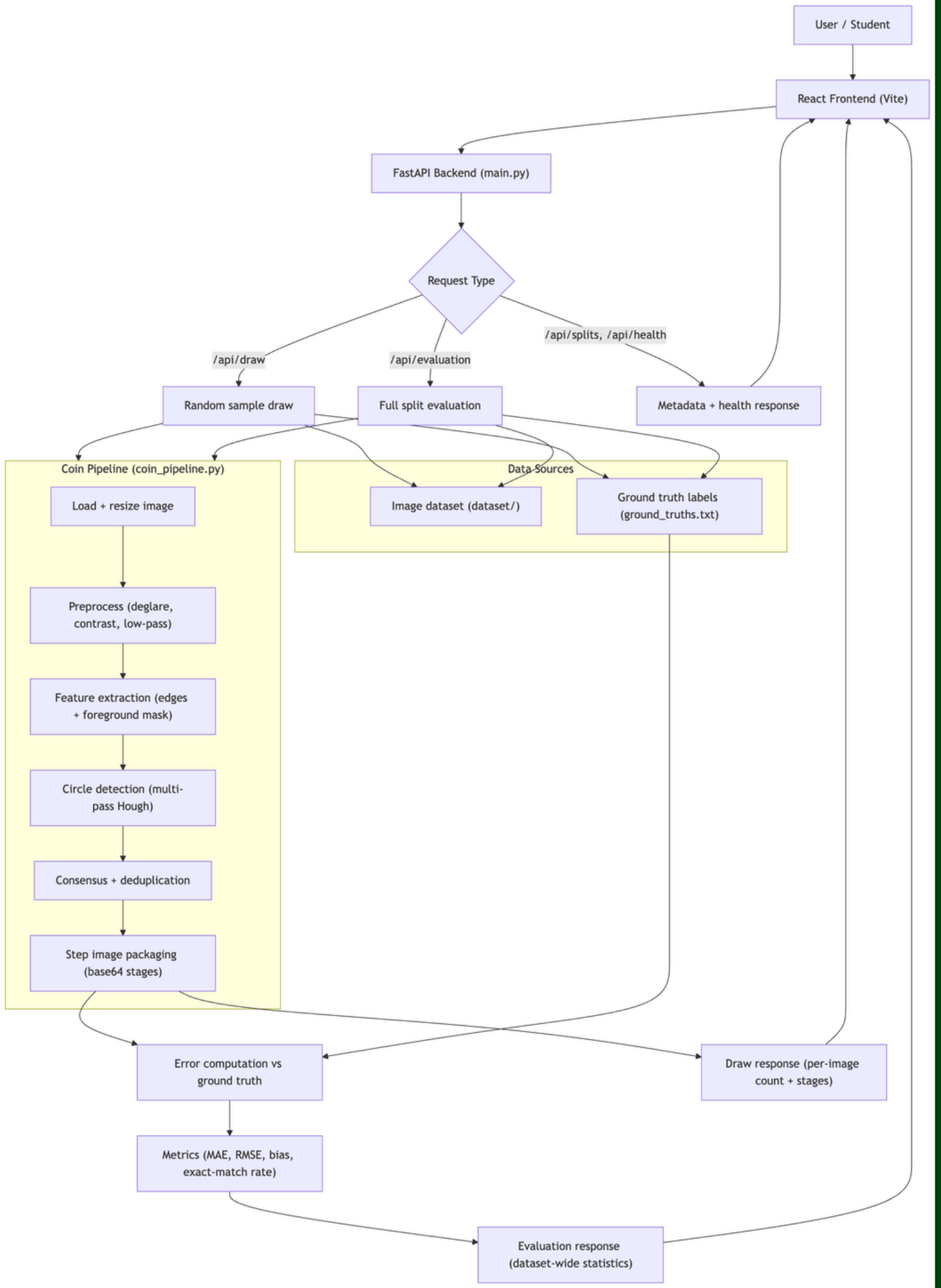
Final Circle Detections

Multiple circle proposals are combined and de-duplicated to produce final coin counts.



Open detailed inspector →

Architecture



Architecture

- **Preprocessing - Filters used**
  - RGB to grayscale
  - glare suppression
  - illumination normalization
  - local contrast enhancement
  - bilateral, FFT low pass, median, Gaussian filters
- **Parameter Tuning**
  - Adaptive parameters by image were used
  - Canny Thresholds - 58th and 86th quantile of gradients
  - Strictness for candidate coins varies based on image
  - Glare ratio, foreground fraction, candidate density, and candidate radiuses are calculated to decide strictness and whether or not to go into a fallback path.

Technical Analysis: Pipeline

- Image is turned gray
- Dark images are lifted and light images are compressed to even out lighting
- Contrast in faint edges is boosted
- Noise and noise-like features of the image are reduced
- Distracting artefacts like background lines are removed through a similar algorithm that detects coins, except for straight lines are detected and the lines are painted out
- A foreground is calculated based on edges detected in the scene
- From the remaining foreground mask, coins are detected
- The image is upscaled to detect smaller coins
- Multiple overlapping detections are mitigated, so that one coin counts as one

Technical Analysis: Challenges

- Extra detections from noisy edges and repeated votes on the same coin
- Tightening filtering parameters led to undercounting as well
- Variability in backgrounds negatively effected the results with false positives; e.g.
  - intersecting straight lines with curves
  - rough surfaces or textured surfaces
- Specular highlights and glare caused uneven detections
- Variability in coin size and coin density led to misses especially for overlapping coins
- Single set of parameters did not work

Challenge Mitigations

- Overlapping detections are voted on
- Glare is detected and painted in with the non-glare scene color
- Splitting the foreground into its own distinct regions helped circle detection
- Pipeline needed to be adaptive according to the image

Robustness

- The dataset itself has a lot of rotated images, and it’s less relevant with coins
- Scale invariance is achieved by scaling up the images with tiny coins.
- Partially obscuring the coins to an extent does not effect the model too much
- High frequency artefacts don’t effect the outcome all that much.
- There was some sensitivity to straight lines observed, though.

Results

Global Dataset Benchmark

SCORED IMAGES	MAE	RMSE	NET ERROR
87/87	**1.85**	2.90	-141

TOTAL ABSOLUTE ERROR	EXACT MATCH RATE
161	26.4%

- On average, the detection is off by 1.85 coins
- There still is an undercount bias
- However, exact matches are low - showcasing the challenges of variation in images.

References

- https://docs.opencv.org/4.x/d5/daf/tutorial_py_histogram_equalization.html
- https://docs.opencv.org/4.x/de/dbc/tutorial_py_fourier_transform.html
- https://docs.opencv.org/4.x/dc/dd3/tutorial_gausian_median_blur_bilateral_filter.html
- https://docs.opencv.org/3.4/d9/db0/tutorial_hough_lines.html
- https://docs.opencv.org/4.x/da/d53/tutorial_py_houghcircles.html
- https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html
- https://docs.opencv.org/4.x/d9/d61/tutorial_py_morphological_ops.html
- https://docs.opencv.org/3.4/d3/dc0/group__imgproc__shape.html
- https://docs.opencv.org/3.4/d4/d73/tutorial_py_contours_begin.html
- https://docs.opencv.org/3.4/d2/dbd/tutorial_distance_transform.html
- https://docs.opencv.org/3.4/df/d3d/tutorial_py_inpainting.html
- https://scikit-image.org/docs/0.25.x/auto_examples/features_detection/plot_blob.html

Thanks!